Autonomous Line Following on the T-Bot

Lab 4 – lane sensing and differential steering on a differential-drive robot

Embedded Real-Time Control laboratory – M.Sc. Control Systems Engineering, University of Padova
Author: Kiavash Rabbani Soudabi (ID 2141822) – Course held by Prof. Alberto Morato – 2025

Purpose

Make the T-Bot follow a high-contrast line autonomously. The line is read by an eight-channel QTR reflectance array behind an SX1509 I^{2}C port expander; the robot is a differential-drive platform (STM32F767, bare-metal C, STM32CubeIDE) whose two motors are closed-loop speed-controlled by the PI controller built in Lab 3.

What I implemented

- **Sensor readout in the control ISR:** the reflectance bank is read over I^{2}C (`REG_DATA_B` of the expander at `0x3E`) inside the `TIM6` period-elapsed callback, so sensing and actuation share the same `TS = 10 ms` frame.
- **Bit-pattern decoding and noise rejection:** `StoreBinary` turns the sensor byte into eight per-sensor flags; `CalculateEsl` drops isolated sensors (a lit sensor with both neighbours dark, and the same rule at the array ends) before any arithmetic.
- **Weighted-centroid lateral position:** each sensor i is weighted by its offset from the array centre, $w_i = (\frac{N-1}{2} - i)\,P$ with $P = 8\,\text{mm}$ and $N = 8$, and the lateral position is

$$e_{sl} = \frac{\sum_i b_i w_i}{\sum_i b_i}, \qquad \sum_i b_i \neq 0.$$

- **Yaw error:** the line offset is turned into a heading error by the lever arm to the vehicle centre, $\hat{\psi} = \arctan(e_{sl}/H)$ with $H = 0.085\,\text{m}$.
- **Steering law and adaptive speed:** the yaw rate command is proportional, $\dot{\psi} = k_\psi \hat{\psi}$ with $k_\psi = 5$; wheel references come from the differential-drive kinematics $v_r = v + \dot{\psi}D/2$, $v_l = v - \dot{\psi}D/2$ with wheelbase $D = 0.165\,\text{m}$. The base speed v is reduced to $0.66\,v$ once $|\hat{\psi}| \geq 0.087\,\text{rad}$ (5°), so the robot slows down in the curves and keeps $0.534\,\text{m/s}$ ($\approx 150\,\text{rpm}$) on straights.
- **Speed loops and actuation:** the wheel references are converted to rpm through the wheel radius ($r = 0.034\,\text{m}$) and tracked by one PI + anti-windup controller per wheel; the control signal in volts is scaled to PWM duty against the measured battery voltage and applied with the lab's `ForwardCoast`/`ForwardBreak` drive modes.
- **Encoder handling across both motors:** the quadrature counters are read with per-motor previous counts and explicit overflow/underflow correction, so the two wheels no longer share state.
- **Instrumentation:** `printf` is retargeted to ITM/SWO for live traces, and the control signals are streamed through the course datalogger for MATLAB analysis.

Results

The robot follows the track on the real T-Bot, holding the line on straights and slowing into the curves; the direction correction comes entirely from the sensor centroid, with no other feedback. Unlike Labs 1–3 this lab had no written report to submit, so this repository documents the implementation directly: the extracted source, this overview and the README's control-law description are the record of the work.

Repository contents

`src/user_code_main.c` (USER CODE sections of my `main.c` – the only part of the file that is mine) · `docs/` (this overview) · `README.md`.

Skills demonstrated

Reflectance-array line sensing over I^{2}C, sensor-pattern filtering, weighted-centroid estimation, differential-drive kinematics and adaptive speed scheduling, cascaded control (steering → wheel speed), fixed-rate real-time scheduling in a timer ISR, dual-motor encoder handling, bare-metal C on ARM Cortex-M.